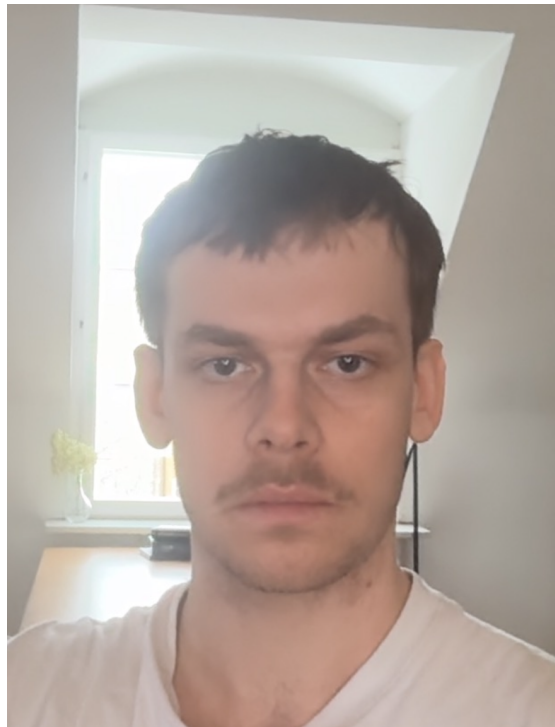


Problem Set #2

Advanced Methods in Applied Statistics (AMAS)

Jeppe Peder Grejs Petersen (rpx928)

February 25th. 2026



1

a

The data is read and transform from the format HH:MM:SS to seconds. I am using Minuit to perform unbinned-likelihood to get the best fit values using the GEV function. I am making sure i am in the support range (see https://en.wikipedia.org/wiki/Generalized_extreme_value_distribution and that σ and ζ is positive. Errors on the parameters are calculated by Minuit, from the Hessian https://scikit-hep.org/iminuit/notebooks/hesse_and_minos.html. I get the best fit values to be

Parameter	Fit Result
$\hat{\mu}$	2960 ± 70
$\hat{\sigma}$	610 ± 50
$\hat{\zeta}$	-0.22 ± 0.09

Table 1: Best-fit parameters with 1σ uncertainties.

I get the negative $\ln$ likelihood to be **1560**. The fitted GEV function is plotted on top if the histogram of the 10000 data points. The GEV function is multiplied with the area of the histogram to scale it.

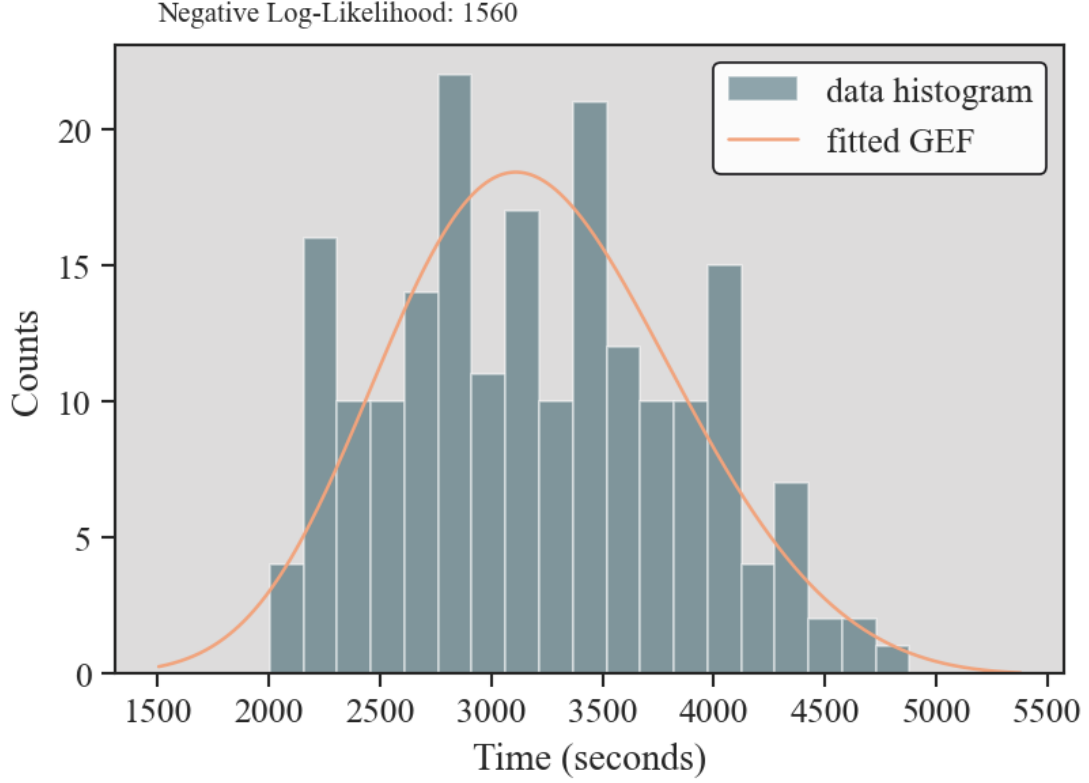


Figure 1: A histogram of the finish-time of the 10k run in seconds. A scaled GEV function, with the best-fit parameters is plotted on top

b

First i find the time of Tanyusha, $t_{tan} = 3747s$, using pandas. Then, i want to integrate the GEV-function, with the best-fit values, from t_{tan} to ∞ , to get the probability of running slower than Tanyusha. I flip the boundaries due to numeric stability, and integrate with `scipy.integrate.quad`:

$$P(t < t_{tan}) = 1 - \int_{1e-5}^{t_{tan}} GEV(x, \hat{\mu}, \hat{\sigma}, \hat{\zeta}) dx = 0.196 \pm 0.043$$

The uncertainty is calculated by doing error-propagation of the uncertainty on the best-fit parameters. The derivatives are calculated numerically.

To calculate the fraction from the data, i loop through all the times, and increment a counter if said time is larger than t_{tan} . Resulting in

$$P_{dat}(t < t_{tan}) = 0.227$$

I assume this is a value without uncertainty

I computed the Z-score, to compare if these to numbers agree statistically.

$$Z = \frac{P(t < t_{tan}) - P_{dat}(t < t_{tan})}{\sigma_{integral}} = 0.6583$$

So well within 1 sigma uncertainty, so one cannot say they do not agree.

Looking at Figure 1, it makes sense that $P(t < t_{tan}) < P_{dat}(t < t_{tan})$, as the area to right of t_{tan} looks to be larger for the histogram.

c

The cdf function is implemented in python the following way

```
def cdf(x,a,b,c):

    integral, _ = scipy.integrate.quad(
        lambda t: GEF(t, a, b, c),
        1e-6,
        x
    )
    return integral
```

And the cdf is shown, along with the data on Figure 2:

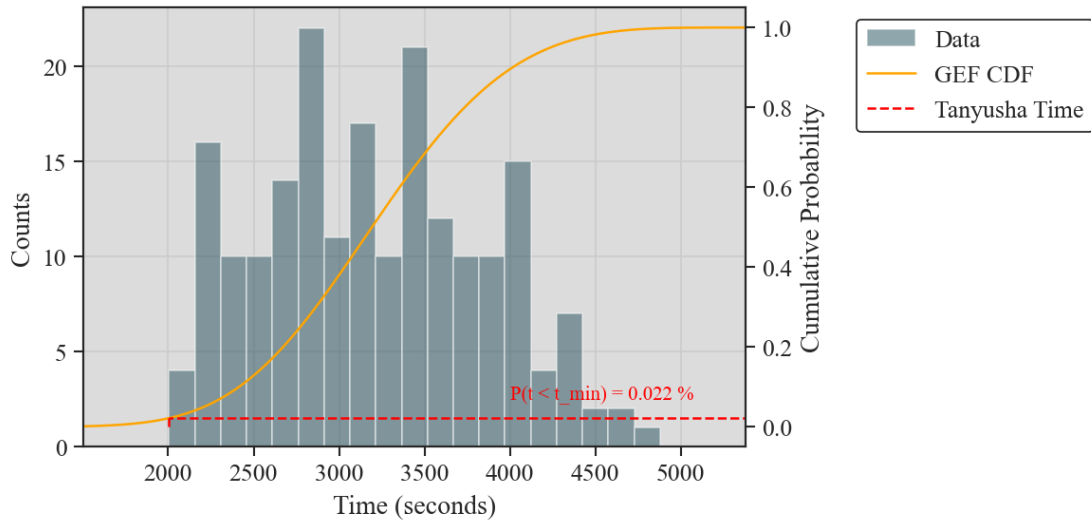


Figure 2: A histogram of the finish-time of the 10k run in seconds. A CDF function, with the best-fit parameters is plotted on top, and this function should be read on the axis-to the right.

The fastest time is found in the data set, and is put into the CDF, to get the probability to get a time faster/lower than the fastest time in the dataset

$$P(t < t_{mindata}) = CDF(t = t_{mindata} | \hat{\mu}, \hat{\sigma}, \hat{\zeta}) = 0.22,$$

2

First the likelihood $f(x)$ is normalized using analytic integration, resulting in $N = 1.6 + 2.66[2.1 - 1.6] = 2.93$. It is given that the likelihood is normalized, as it is a pdf. The posterior is normalized by doing numerical integration of the prior x likelihood, in the range $x \in [0, 2.1]$ (same as $x \in [0, 4]$ as prior = 0 in $[2.1, 4]$) .

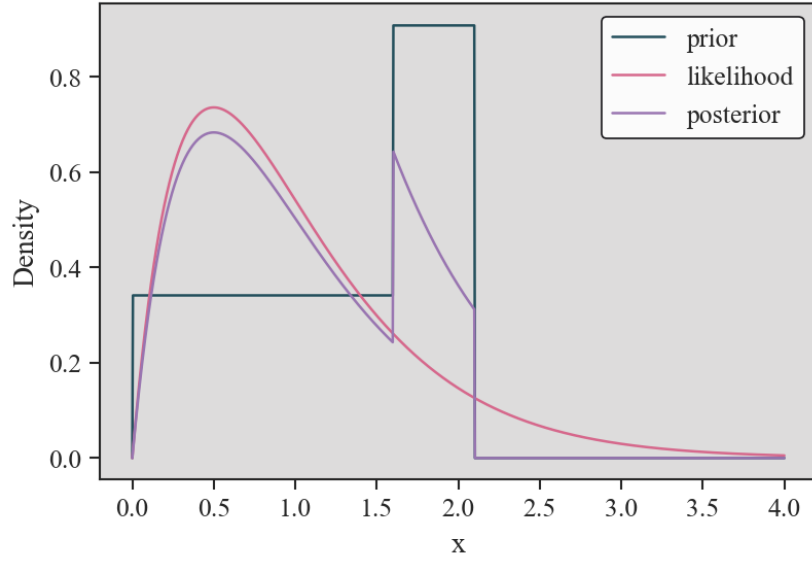


Figure 3: The normalized posterior, prior and likelihood.

The most likely value of x , $\hat{x}$ is, where the posterior is maximized, and is

$$\hat{x} = 0.50$$

Resulting in a normalized Bayesian posterior of

$$g(\hat{x}) = 0.68$$

3

a

I created my grid using `np.meshgrid`, so that i could plot the positions in physical units. I have discretized the problem so each grid cell is $\approx 20m$. To take a step I draw a random angle, and update the previous position with this step. To take the edge-case into account, for each proposed step, i test wether it falls outside the circle. If that is the case i decrease the step size until it falls within the circle.

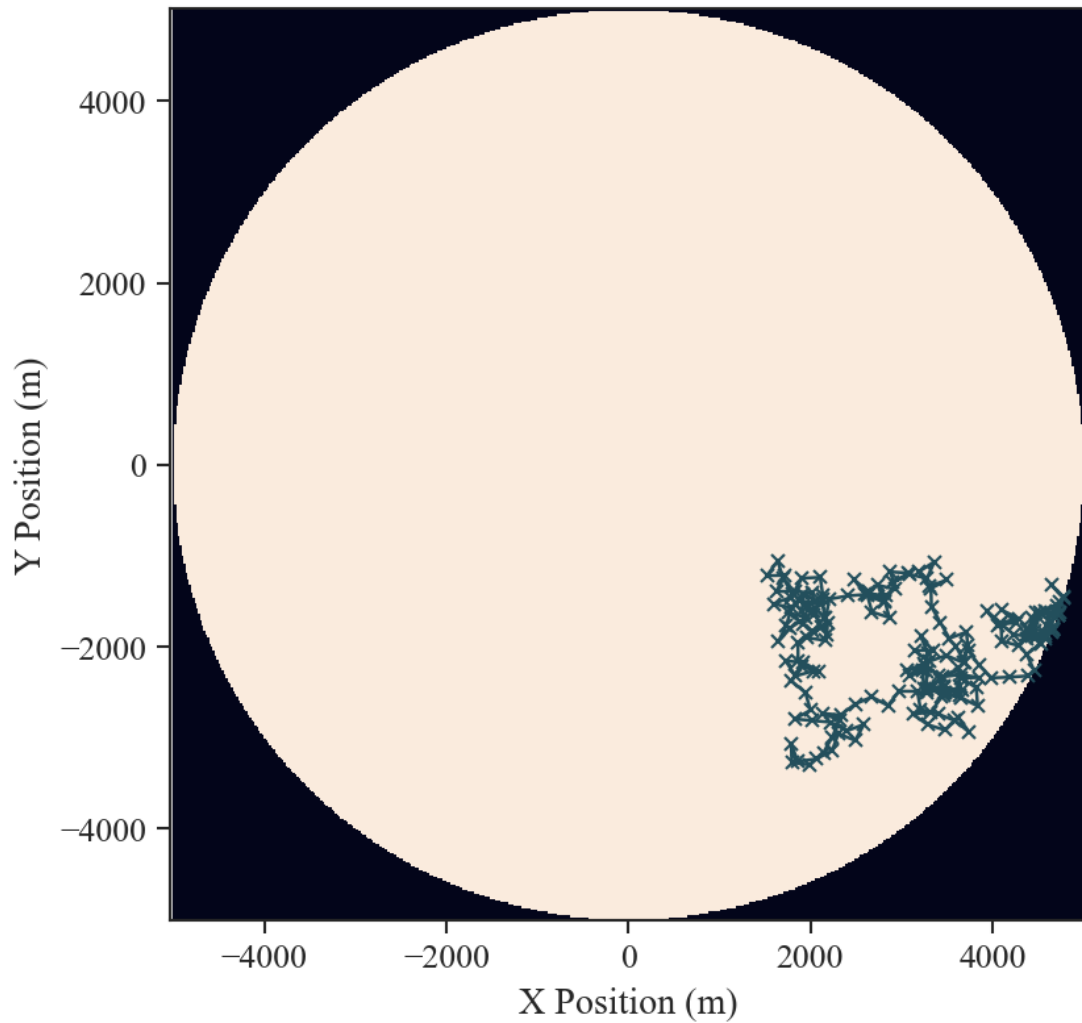


Figure 4: A crab moving for 200 days. Note how it handles the edge case correctly.

b

I use the same logic as before, but if a crab steps outside, I return a variable, `run` set to `False`, and a single pseudo-experiment, is ran the following way:

```
def experiment():
    lens = 0
    pos = init_pos
    run = True
    while run :
        new_pos, leng, run = take_step(pos)
        lens += leng
        pos = new_pos
    return lens
```

This is ran 501 times, to produce the following histogram of distributions of the distance it took the crabs to reach the edge.

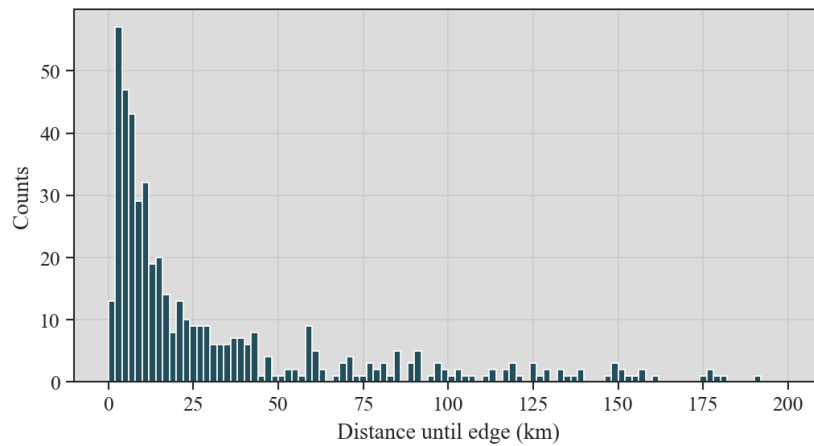


Figure 5

c

To solve this problem, some functions needs to be chanced. The `take_step()` function is vectorized, so it takes positions of shape $(N_{crabs}, 2)$. The same logic as explained before still applies, just handling multiple crabs at the same time. This also ensures once can use the vectorization capabilities of `numpy`. A distance matrix is created, and the diagonal is set to $1e5$, so the distance between any pair of crabs easily can be calculated. All the pairs (can be larger than 2) is sorted by distance, and if the mass is 0 (meaning dead crab), it is skipped. Then the probability is calculated, implemented with the function

```
def prop(M,m):
    return M/(M + m)
```

Where M is the mass of the larger crab and m is the mass of the smaller crab. Then the probability is compared with a number drawn from a uniform distribution, u , and the larger crab wins, if the probability is larger than u . The mass of the smaller crab is added to the larger crab, and it itself is set to 0. The position of the dead crab is set to `np.nan`. Then a new step is taken, and the process is repeated. The resulting distributions of masses of the largest crab and the number of crabs alive is shown on Figure 6

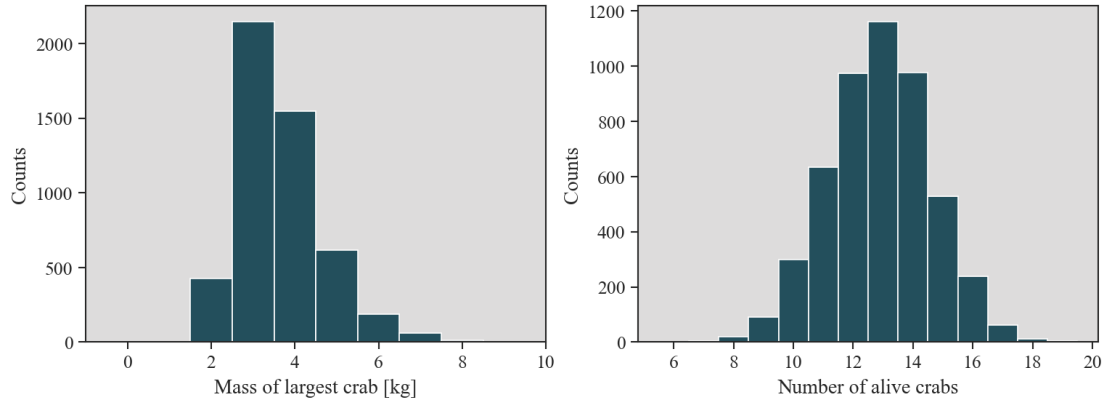


Figure 6: Left: The distribution of mass of the largest Crab after 200 days. Right: The distribution of the Number of crabs alive after 200 days. In both cases 5000 experiments is ran

From the histogram it can be easily read that **the mostly likely number of individual crabs that remain is 13** and **the most likely mass of the largest crab is 3kg**

d

To calculate the distribution of days it takes until only 10 crabs remain alive, the logic is explained in 3c. The simulation however terminates when the amount of crabs alive is 10, instead of stopping after 200 days.

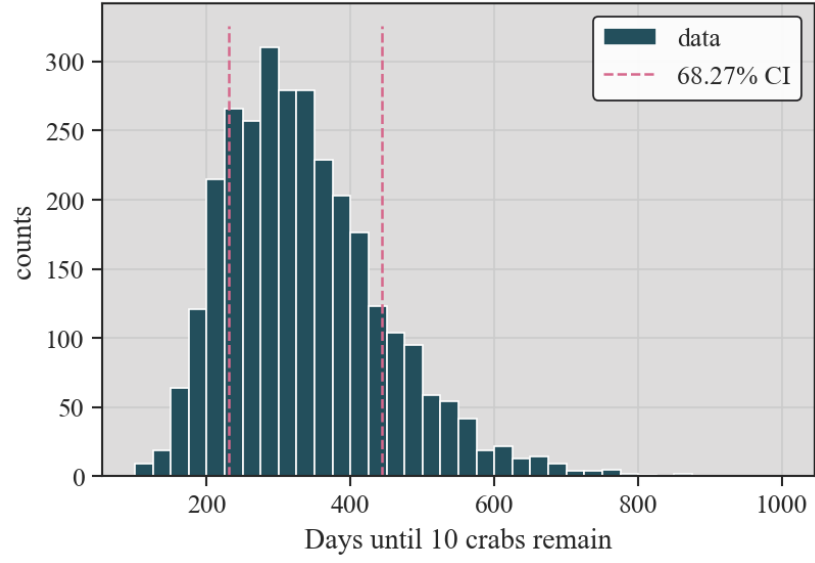


Figure 7: The distribution of days it takes until only 10 crabs, with the 1σ confidence interval

To calculate the 1σ intercalate, the function `np.percentile` is used. **The 1σ confidence interval is calculated to be $[230, 443]$**